

Questão 07 - Runbook para alerta recorrente

Prompt (aplicando R-I-S-E)

[ROLE]

Você é um SRE Sênior da Hill Valley Tech, especialista em Kubernetes/EKS e em elaboração de runbooks operacionais. Escreva para dois públicos: você é quem documenta, e quem vai executar é um plantonista genérico que pode não conhecer profundamente o Chronos — o runbook precisa ser autossuficiente.

[INPUT]

Alerta recorrente do Beacon (observabilidade):

"[CRITICAL] High memory usage on Chronos API pods (>85% for 10min)"

Frequência: ~4x por semana. Tempo médio de resolução hoje: 30-40min, com alta variação por falta de procedimento documentado.

Ambiente:

- Chronos roda no EKS, namespace production, 6 réplicas, HPA configurado (min 4, max 12, CPU target 70% — atenção: HPA é baseado em CPU, não em memória, então não reage automaticamente a esse alerta)
- Deploy via Argo CD, repositório hvt/chronos-api
- Dependências diretas: Ledger (PostgreSQL) e Reactor (filas SQS)
- Métricas em /metrics, logs centralizados no Beacon, dashboards em Grafana
- Ferramentas disponíveis: kubectl, aws cli, argocd cli
- Canal de plantão: #oncall-chronos (Slack)
- Escalação: @chronos-core, SLA 15min (horário comercial) / 30min (fora)

[STEPS]

Estruture o runbook com as seguintes etapas, cada uma contendo: objetivo do passo, comando(s) específico(s) a executar, e o que verificar no resultado antes de avançar para o próximo passo.

1. Confirmação do alerta e escopo do impacto (quantos pods afetados, se é generalizado ou pontual)
2. Diagnóstico inicial: uso de memória por pod, histórico recente (o problema é gradual — memory leak — ou um pico súbito?)
3. Verificação de causa provável relacionada às dependências (Ledger e Reactor podem gerar acúmulo de memória no Chronos, ex.: conexões presas, fila crescendo)
4. Verificação se houve deploy recente via Argo CD que possa ter introduzido o problema
5. Ação de mitigação imediata (restart controlado dos pods afetados, sem causar indisponibilidade, considerando o HPA)
6. Critérios objetivos e mensuráveis para decidir escalar para @chronos-core em vez de continuar tentando mitigar sozinho
7. Critério objetivo para considerar o incidente encerrado

[EXPECTATION]

O runbook deve permitir que qualquer plantonista, mesmo sem conhecimento prévio profundo do Chronos, reduza o tempo médio de resolução (hoje 30-40min) de forma consistente, seguindo passos objetivos sem depender de tribal knowledge. Cada etapa deve ter um resultado esperado claro e verificável ("o que você deve ver na tela para saber que pode avançar"), e os critérios de escalação e encerramento devem ser objetivos (números, condições claras), nunca subjetivos como "se achar que está grave".

[FORMATO DE SAÍDA]

Documento estruturado em Markdown, no formato de runbook (título, contexto resumido do alerta, e as 7 etapas numeradas com sub-itens: Objetivo, Comando, Verificação esperada), finalizando com as seções "Critérios de Escalação" e "Critério de Encerramento".

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Justificativa do uso

Modelo de referência do curso. Forte aderência a prompts estruturados em múltiplos blocos, raciocínio encadeado consistente e geração de artefatos técnicos (YAML, HCL, bash, SQL) com alto grau de precisão. Escolhido como baseline por permitir avaliação consistente dos frameworks ao longo de todo o desafio.

Output gerado

Runbook: High Memory Usage — Chronos API Pods

****Alerta:**** `[CRITICAL] High memory usage on Chronos API pods (>85% for 10min)`

****Origem:**** Beacon | ****Canal:**** #oncall-chronos | ****Escalação:**** @chronos-core

1. Confirmação do alerta e escopo do impacto

****Objetivo:**** confirmar que o alerta reflete a realidade e entender se é pontual (1-2 pods) ou generalizado (maioria dos pods).

****Comando:****

kubectl top pods -n production -l app=chronos-api --sort-by=memory

****Verificação esperada:**** identifique quantos pods estão acima de 85% do limit de memória configurado. Se apenas 1 pod está afetado, é provável que seja isolado (ver Passo 2). Se $\geq 50\%$ dos pods estão afetados, trate como sistêmico e vá direto ao Passo 4.

2. Diagnóstico inicial: padrão de consumo

****Objetivo:**** diferenciar memory leak gradual de pico súbito.

****Comando:****

```
kubectl get pods -n production -l app=chronos-api -o wide
```

Verificar dashboard Grafana "Chronos - Memory Usage" (últimas 2h)

****Verificação esperada:**** se a curva de memória sobe de forma constante e linear ao longo de horas → leak (candidato a restart + investigação). Se o pico é abrupto e coincide com um horário específico → investigar causa externa (Passo 3).

3. Verificação de causa relacionada a dependências

****Objetivo:**** checar se Ledger ou Reactor estão gerando acúmulo de memória no Chronos (ex.: conexões de banco não liberadas, fila SQS crescendo e sendo bufferizada em memória).

****Comando:****

```
kubectl logs -n production -l app=chronos-api --since=30m | grep -i "connection\|timeout\|queue"
```

```
aws sqs get-queue-attributes --queue-url <reactor-queue-url> \
--attribute-names ApproximateNumberOfMessages
```

****Verificação esperada:**** volume anormal de mensagens na fila SQS (acima do padrão histórico) ou logs com erros de timeout/conexão indicam causa externa. Se a fila está saudável e não há erros de conexão, o problema é provavelmente interno ao Chronos.

4. Verificação de deploy recente via Argo CD

****Objetivo:**** identificar se uma mudança de código recente introduziu o problema.

****Comando:****

argocd app get chronos-api

argocd app history chronos-api

****Verificação esperada:**** se houve deploy nas últimas 24h e o timing coincide com o início do consumo anormal de memória, é forte candidato a causa raiz — considere rollback (Passo 5b) em vez de apenas restart.

5. Ação de mitigação imediata

****Objetivo:**** aliviar o consumo de memória sem causar indisponibilidade.

****5a. Se NÃO houve deploy recente suspeito — restart controlado:****

kubectl rollout restart deployment/chronos-api -n production

kubectl rollout status deployment/chronos-api -n production

****5b. Se houve deploy recente suspeito (Passo 4) — rollback:****

argocd app rollback chronos-api <revision-anterior>

****Verificação esperada:**** `rollout status` deve retornar

"successfully rolled out". Nos minutos seguintes, o uso de memória

(Passo 1) deve cair para abaixo de 70%. Como há 6 réplicas com HPA (min 4), o restart é rolling por padrão e não deve causar indisponibilidade — confirme que o número de pods `Ready` se mantém ≥ 4 durante todo o processo.

Critérios de Escalação (objetivos)

Escale imediatamente para **@chronos-core** se qualquer um dos critérios abaixo for verdadeiro:

- Após o restart/rollback (Passo 5), o uso de memória volta a ultrapassar 85% em menos de 15 minutos
- $\geq 50\%$ dos pods estão afetados simultaneamente (indício de causa sistêmica, não isolada)
- A fila SQS do Reactor (Passo 3) está com crescimento superior a 3x a média histórica e não estabiliza
- Já se passaram 20 minutos desde o início do diagnóstico sem identificação de causa provável
- SLA de resposta: 15min em horário comercial, 30min fora — abra o incidente no #oncall-chronos citando @chronos-core com o resumo dos Passos 1-4 já executados

Critério de Encerramento

O incidente pode ser considerado encerrado quando **todas** as condições abaixo forem verdadeiras:

- Uso de memória de todos os pods do Chronos está abaixo de 70% (target do HPA) por pelo menos 15 minutos consecutivos

- Nenhum pod em `CrashLoopBackOff` ou `Restarting`
- `kubectl get pods -n production -l app=chronos-api` mostra todos os pods em status `Running` e `Ready`
- Causa raiz identificada e registrada (mesmo que a correção definitiva fique para um follow-up posterior)

Justificativa — como R-I-S-E aparece no prompt

Componente	Onde aparece	Por quê
Role	"SRE Sênior... especialista em runbooks... escreva para um plantonista que pode não conhecer profundamente o Chronos"	Define a dupla perspectiva necessária: expertise técnica de quem escreve + acessibilidade para quem executa sob pressão, num incidente real
Input	Texto do alerta, frequência histórica, topologia do ambiente (EKS, HPA, Argo CD, dependências, ferramentas, canal, SLA de escalação)	Fornece todos os dados que o runbook precisa referenciar — sem isso a IA geraria um runbook genérico de "high memory" sem os comandos e nomes específicos do ambiente real da Hill Valley Tech
Steps	As 7 etapas numeradas, cada uma exigindo objetivo + comando + verificação	É o núcleo procedural — obriga a IA a produzir algo executável (comandos reais de kubectl/argocd/aws cli), não apenas uma descrição textual do que fazer
Expectation	"reduzir o tempo médio de resolução... critérios objetivos, nunca subjetivos como 'se achar que está grave'"	É o que transforma o output de "lista de sugestões" em runbook de verdade — força critérios mensuráveis de escalação/encerramento, atacando diretamente a dor citada por Lorraine no enunciado (variação alta por falta de procedimento)